

Here is a **backend API spec**, ordered by **priority**, including **method**, **path**, **params**, **brief description**, **auth (Clerk)**, and **data flow** for each endpoint. Clerk is only required for reviewer/admin actions; public/reporting flows stay anonymous. [globaleaks+2](#)

Priority 1 – Core Reporting & Artwork

1. Create Report (Form submit)

Method & Path

- `POST /api/reports`

Auth (Clerk)

- Public (no auth); anonymous reporter.

Description

- Accepts a new whistleblower report, encrypts the secret payload, generates identifiers, and returns `reportId`, `artId`, and a one-time `decodeKey`.

Request body

```
json
{
  "category": "corporate",           // required, enum/string
  "locationContext": "Large bank...", // required, short text
  "description": "Secret details...", // required, long text
  "externalLink": "https://...",    // optional, null allowed
  "sentimentHint": null              // optional, reserved for
  future AI color
}
```

Response body (200/201)

```
json
{
  "reportId": "R123ABC",
  "artId": "R123ABC",               // same as reportId for MVP
}
```

```
"decodeKey": "X7K-9PQ-4LM", // only returned once
"createdAt": "2026-01-08T06:30:00.000Z",
"category": "corporate"
}
```

Data flow

1. Validate input schema and lengths.[github](#)
 2. Generate `reportId` (nanoid/UUID).
 3. Generate random `decodeKey` (secure random).
 4. Compute `decodeKeyHash = SHA256(decodeKey)`.[github](#)
 5. Build plaintext payload JSON { `description`, `externalLink` }.
 6. Encrypt payload with server key (AES-GCM or similar) → `encryptedPayload`,
`iv`.[whistleblowing](#)
 7. Derive artwork seed from `hash(reportId + ":" + category)` and optionally store.
 8. Insert DB row: `id`, `category`, `location_context`, `encrypted_payload`,
`encryption_iv`, `decode_key_hash`, `created_at`, `status='new'`,
`reviewer_note=null`.
 9. Return `reportId`, `artId`, `decodeKey`, `createdAt`, `category`.
-

Priority 2 – Public Gallery & Artwork

2. List Artworks for Gallery

Method & Path

- `GET /api/artworks`

Auth (Clerk)

- Public.

Description

- Returns paginated list of artwork metadata for the public gallery (no decrypted content).

Query params

text

```
page=1          // optional, default 1
pageSize=20     // optional, default 20
```

Response body

json

```
{
  "items": [
    {
      "artId": "R123ABC",
      "createdAt": "2026-01-08T06:30:00.000Z",
      "category": "corporate"
    }
  ],
  "page": 1,
  "pageSize": 20,
  "total": 42,
  "hasMore": true
}
```

Data flow

1. Parse pagination params with defaults.
 2. Query DB for `reports` ordered by `created_at DESC, LIMIT/OFFSET`. [github](#)
 3. Map rows $\rightarrow$ `{ artId: id, createdAt, category }`.
 4. Compute `total` via `COUNT(*)`.
 5. Calculate `hasMore = (page * pageSize < total)`.
 6. Return list + pagination metadata.
-

3. Get Single Artwork Metadata (optional but useful)

Method & Path

- `GET /api/artworks/:artId`

Auth (Clerk)

- Public.

Description

- Fetches minimal metadata for a single artwork, e.g., for confirmation page reload.

Path params

text

```
:artId // same as reportId
```

Response body

json

```
{
  "artId": "R123ABC",
  "createdAt": "2026-01-08T06:30:00.000Z",
  "category": "corporate"
}
```

Data flow

1. Lookup report by `id = artId`.
 2. If not found, 404.
 3. Return `{ artId, createdAt, category }`.
-

4. Generate Artwork SVG (optional server-side)

Method & Path

- `GET /api/artworks/:artId/svg`

Auth (Clerk)

- Public.

Description

- Returns SVG image for given `artId`, using deterministic pattern from report data.

Response

- `Content-Type: image/svg+xml`
- Raw SVG string (no JSON).

Data flow

1. Lookup report by `id = artId` to get `category`.
 2. Derive seed = hash(`reportId + ":" + category`).[okta+1](#)
 3. Use seeded PRNG to generate shapes/colors → SVG string.
 4. Send SVG as response body.
-

Priority 3 – Trusted Decode Flow (Public but key-protected)

5. Decode Report by ID + Key

Method & Path

- `POST /api/decode`

Auth (Clerk)

- Public (no Clerk), but requires valid key.

Description

- Validates decode key for given reportId; on success, decrypts and returns full report content.

Request body

```
json
{
  "reportId": "R123ABC",
  "decodeKey": "X7K-9PQ-4LM"
}
```

Response body (success)

```
json
{
```

```
"reportId": "R123ABC",
"category": "corporate",
"locationContext": "Large bank...",
"description": "Full decrypted description...",
"externalLink": "https://...",
"createdAt": "2026-01-08T06:30:00.000Z",
"status": "new"
}
```

Response body (error, 400/404)

```
json
{
  "error": "Cannot decode – ID/key mismatch."
}
```

Data flow

1. Validate request schema.
2. Lookup report by `id = reportId`.
3. If not found → respond with generic error.
4. Compute `hash(decodeKey)` and compare with `decode_key_hash`. [github](#)
5. If mismatch → generic error “Cannot decode – ID/key mismatch.” (do not specify which part failed).
6. If match:
 - Decrypt `encrypted_payload` using `encryption_iv` and server key. [whistleblowing](#)
 - Parse JSON → `{ description, externalLink }`.
 - Return full content + metadata (no decode key or hash).

Priority 4 – Reviewer Auth (Clerk)

Reviewer console should use **Clerk** for authentication. Backend verifies Clerk JWT/session on protected routes. [clerk+2](#)

Assume:

- Frontend uses Clerk for sign-in (e.g., `/sign-in`, `/sign-up` hosted by Clerk).

- Backend uses Clerk middleware/SDK to protect `/api/reviewer/*` routes and extract `userId`.[github+1](#)

6. Get Current Reviewer Profile

Method & Path

- `GET /api/reviewer/me`

Auth (Clerk)

- **Required** – use Clerk middleware (`ClerkExpressRequireAuth` or equivalent).[clerk](#)

Description

- Simple check that the requester is an authenticated reviewer; returns their id/email. Used by frontend to verify session.

Response body

```
json
{
  "userId": "user_123",
  "email": "reviewer@example.com"
}
```

Data flow

1. Clerk middleware verifies token/session.[github+1](#)
2. Extract user information from Clerk context (userId, primaryEmail).
3. Optionally check role/allowlist if needed (e.g., only some Clerk users are “reviewers”).
4. Return basic identity fields.

No explicit `/login` or `/logout` API needed if using Clerk; sign-in/out handled by Clerk frontend components.

Priority 5 – Reviewer Console: List & Detail

All `/api/reviewer/*` endpoints are **protected with Clerk**:

- Use Clerk middleware to **require auth**.[stackoverflow+1](#)
- Optionally verify the user is in a “reviewer” list/role.

7. List Reports for Reviewer Console

Method & Path

- `GET /api/reviewer/reports`

Auth (Clerk)

- **Required** (reviewer only).

Description

- Returns paginated list of reports with minimal metadata for the console queue.

Query params

text

```
status=new          // optional: new|in_review|closed; default: all
page=1              // optional
pageSize=20         // optional
```

Response body

json

```
{
  "items": [
    {
      "reportId": "R123ABC",
      "createdAt": "2026-01-08T06:30:00.000Z",
      "category": "corporate",
      "status": "new"
    }
  ],
  "page": 1,
  "pageSize": 20,
  "total": 10,
  "hasMore": false
}
```

Data flow

1. Clerk middleware verifies user; get `userId`.[clerk](#)
 2. Optionally check if `userId` is allowed reviewer (env allowlist, role, etc.).
 3. Parse `status`, `page`, `pageSize`.
 4. Build DB query:
 - If `status` provided → `WHERE status = $status`.
 - Else return all.
 5. Order by `created_at` DESC.
 6. Apply pagination (`LIMIT`, `OFFSET`).
 7. Map to `{ reportId, createdAt, category, status }`.
 8. Return list + pagination metadata.
-

8. Get Full Decoded Report (Reviewer Detail)

Method & Path

- `GET /api/reviewer/reports/:reportId`

Auth (Clerk)

- **Required** (reviewer only).

Description

- Returns full decrypted report details for a specific report, only to authenticated reviewers.

Path params

text
:reportId

Response body

json
{
 "reportId": "R123ABC",
 "createdAt": "2026-01-08T06:30:00.000Z",

```
"category": "corporate",
"locationContext": "Large bank...",
"description": "Full decrypted description...",
"externalLink": "https://...",
"status": "new",
"reviewerNote": "Internal note or empty string"
}
```

Data flow

1. Clerk middleware verifies user; confirm reviewer.[github+1](#)
 2. Lookup report by `id = reportId`.
 3. If not found, 404.
 4. Decrypt `encrypted_payload` with server key and `encryption_iv`.[whistleblowing](#)
 5. Parse payload JSON → `description`, `externalLink`.
 6. Return full report data including `reviewer_note` and `status`.
 7. Do **not** expose `decodeKeyHash` or any crypto internals.
-

9. Update Report Status & Internal Note

Method & Path

- `PATCH /api/reviewer/reports/:reportId`

Auth (Clerk)

- **Required** (reviewer only).

Description

- Allows reviewer to change status and add/update a short internal note. No message goes back to reporter.

Path params

text
`:reportId`

Request body

```
json
{
  "status": "in_review",           // required; one of "new" |
  "in_review" | "closed"
  "reviewerNote": "Short note..." // optional (string or null)
}
```

Response body

```
json
{
  "reportId": "R123ABC",
  "status": "in_review",
  "reviewerNote": "Short note..."
}
```

Data flow

1. Clerk middleware verifies user; confirm reviewer.[clerk](#)
 2. Validate `status` against allowed values; trim and limit `reviewerNote`.
 3. Update DB row: set `status` and `reviewer_note`.
 4. Optionally record `updated_at` timestamp (for future audit).
 5. Return updated `reportId`, `status`, `reviewerNote`.
-

Priority 6 – Optional / Supporting APIs

These are nice-to-have but not strictly required for core demo.

10. Health Check

Method & Path

- `GET /api/health`

Auth (Clerk)

- Public.

Description

- Simple readiness/status check for deployment.

Response

```
json
{
  "status": "ok",
  "timestamp": "2026-01-08T06:30:00.000Z"
}
```

11. (Optional) Reviewer-only Artwork Listing with Extra Fields

Method & Path

- `GET /api/reviewer/artworks`

Auth (Clerk)

- Reviewer only.

Description

- Same as public `/api/artworks`, but could include extra metadata (status, internal tags) for future advanced console.
-

Clerk Integration Summary (Backend)

- Use Clerk middleware for Node/Express or your framework's integration:
 - Protect reviewer routes:
 - `/api/reviewer/*`
 - Leave public routes open:
 - `/api/reports`
 - `/api/artworks*`
 - `/api/decode`
- For Express, e.g.: `ClerkExpressRequireAuth()` on `/api/reviewer/*`. [clerk](#)

- In handlers, access `req.auth.userId` or similar to identify the reviewer.[github+1](#)
-

If you want next, can provide a **concise OpenAPI-style table** or JSON for all these endpoints that you can drop into Swagger/Postman.

1. <https://www.globaleaks.org>
2. <https://clerk.com/blog/securing-node-express-apis-clerk-react>
3. <https://hushline.app>
4. <https://github.com/invzbl3/WhistleblowerApplication>
5. <https://www.whistleblowing.software/en/>
6. <https://www.okta.com/identity-101/steganography/>
7. <https://manytools.org/hacker-tools/steganography-encode-text-into-image/>
8. <https://clerk.com/blog/next13-api-routes-2>
9. <https://stackoverflow.com/questions/78571655/protected-routes-and-public-routes-with-the-new-clerk-update>
10. https://github.com/cnndabbler/clerk_fastapi
11. <https://whistleblowersoftware.com>
12. <https://www.getapp.com/hr-employee-management-software/whistleblowing/f/api/>
13. <https://cdn.dam.fixit-holding.com/assets/api/e75ca4ca-9db9-47cf-a1c5-5d5af3b3b733/original/Wistleblower-platform-of-FIXIT-GRUPPE.pdf>
14. <https://github.com/globaleaks/GlobaLeaks/blob/main/backend/globaleaks/rest/api.py>
15. <https://diss-co.tech/home/whistleblowing-software>
16. <https://www.whistleblowersecurity.com/press/onecomplys-360-compliance-platform-integrates-whistleblower-security-alerts>
17. <https://help.mabl.com/hc/en-us/articles/19083859290132-Reviewing-API-tests>
18. <https://www.faceup.com/en>